



曦云[®] 系列通用 GPU

MCCL 编程指南

PG-23004-020-F5_V06

2025-04-21

声明

版权所有 ©2023-2025 沐曦集成电路（上海）股份有限公司。保留所有权利。

本文档中呈现的信息属于沐曦集成电路（上海）股份有限公司和/或其附属公司（以下统称为“沐曦”），非经沐曦事先书面许可，任何实体或个人均不得获得本文档的副本，且无权以任何方式处理本文档，包括但不限于使用、复制、修改、合并、出版、发行、销售或传播本文档的部分或全部。

本文档内容仅供参考，不提供任何形式的、明示或暗示的保证，包括但不限于对适销性、适用于任何目的和/或不侵权的保证。在任何情况下，沐曦均不对因本文档引起的、由本文档造成的、或与之相关的任何索赔、损害或其他责任负责。

沐曦保留自行决定随时更改、修改、添加或删除本文档的部分或全部的权利。沐曦保留最终解释权。

沐曦、MetaX 和其他沐曦图标是沐曦的商标。本文档中提及的所有其他商标和商品名称均为其各自所有者的财产。

目录

1 概述	1
2 简介	2
2.1 系统架构	2
3 编程模型	4
3.1 通讯器管理	4
3.1.1 mcclGetVersion	4
3.1.2 mcclGetUniqueld	4
3.1.3 mcclCommInitRank	4
3.1.4 mcclCommInitAll	4
3.1.5 mcclCommDestroy	5
3.1.6 mcclCommAbort	5
3.1.7 mcclGetErrorString	5
3.1.8 mcclCommGetAsyncError	5
3.1.9 mcclCommCount	5
3.1.10 mcclCommMcDevice	5
3.1.11 mcclCommUserRank	6
3.1.12 示例	6
3.1.12.1 单进程	6
3.1.12.2 多进程	6
3.2 集合通讯	7
3.2.1 mcclAllReduce	7
3.2.2 mcclBroadcast	7
3.2.3 mcclReduce	7
3.2.4 mcclAllGather	8
3.2.5 mcclReduceScatter	8
3.2.6 mcclAllToAll	8
3.2.7 mcclAllToAllv	8
3.2.8 mcclAllToAlld	9
3.2.9 示例	9
3.3 组调用	10
3.3.1 mcclGroupStart	11
3.3.2 mcclGroupEnd	11
3.3.3 示例	11
3.4 点对点通讯	11
3.4.1 mcclSend	11
3.4.2 mcclRecv	12
3.4.3 示例	12
3.5 数据类型	12

3.6	返回值类型	13
3.7	归约操作类型	13
4	环境变量	14
4.1	推荐环境变量	14
4.1.1	多机环境	14
4.1.2	极致性能	14
4.1.3	Arm 服务器	15
4.1.4	异构集群	15
4.1.5	虚拟化场景	15
4.1.6	C500X 机型	16
4.1.7	阡视服务器	16
4.1.8	大模型通信日志	16
4.2	环境变量说明	16
4.2.1	功能相关	16
4.2.1.1	MCCL_DISABLE_OPTIC_LINK	16
4.2.1.2	MCCL_FAST_WRITE_BACK	16
4.2.1.3	MCCL_EARLY_WRITE_BACK	17
4.2.1.4	MCCL_GROUP_WRITE_BACK	17
4.2.1.5	MCCL_DISABLE_MULTI_NODE_FABRIC	17
4.2.1.6	MCCL_BUFFSIZE	17
4.2.1.7	MCCL_DISABLE_CACHEABLE_BUFFER	18
4.2.1.8	MCCL_THRESHOLD_TO_USE_CACHEABLE_BUFFER	18
4.2.1.9	MCCL_THRESHOLD_TO_USE_CACHEABLE_BUFFER_P2P	18
4.2.1.10	MCCL_PCIE_BUFFER_MODE	18
4.2.1.11	MCCL_TUNING_MODEL	19
4.2.1.12	MCCL_PROTO	19
4.2.1.13	MCCL_ALGO	19
4.2.1.14	MCCL_DMABUF_ENABLE	19
4.2.1.15	MCCL_EXT_CCL_ENABLE	20
4.2.1.16	MCCL_HC_PLUGIN	20
4.2.1.17	MCCL_RINGS	20
4.2.2	性能相关	20
4.2.2.1	MCCL_ENABLE_FC	20
4.2.2.2	MCCL_ENABLE_FC8_OAM	21
4.2.2.3	MCCL_FC_BYTE_LIMIT	21
4.2.2.4	MCCL_FC_MAX_BLOCKS	21
4.2.2.5	MCCL_FC_DISABLE_REMOTE_READ	21
4.2.2.6	MCCL_FC_BYTE_LIMIT_DRAGONFLY	21
4.2.2.7	MCCL_FC_MTLK_BLOCKS	22
4.2.2.8	MCCL_LIMIT_RING_LL_THREADTHRESHOLDS	22
4.2.2.9	MCCL_CROSS_NIC	22
4.2.2.10	MCCL_MIN_NCHANNELS	22
4.2.2.11	MCCL_MAX_NCHANNELS	23
4.2.2.12	MCCL_RING_TP8_MODE	23

4.2.2.13	MCCL_NET_DISABLE_INTRA	24
4.2.2.14	MCCL_PXN_DISABLE	24
4.2.2.15	MCCL_MIN_P2P_NCHANNELS	24
4.2.2.16	MCCL_MAX_P2P_NCHANNELS	24
4.2.2.17	MCCL_P2P_NCHANNELS	24
4.2.2.18	MCCL_TUNING_FILE	25
4.2.2.19	MCCL_TOPO_FILE	25
4.2.2.20	MCCL_TOPO_DUMP_FILE	25
4.2.2.21	MCCL_P2P_DISABLE	25
4.2.2.22	MCCL_P2P_LEVEL	25
4.2.2.23	MCCL_DF16_RINGS	26
4.2.2.24	MCCL_IGNORE_CPU_AFFINITY	26
4.2.2.25	MCCL_RUNTIME_CONNECT	27
4.2.2.26	MCCL_HFC_EP4_KERNEL_LIMIT	27
4.2.2.27	MCCL_HFC_EP8_KERNEL_LIMIT	27
4.2.3	网络相关	27
4.2.3.1	MCCL_SHM_DISABLE	27
4.2.3.2	MCCL_IB_GID_INDEX	28
4.2.3.3	MCCL_IB_DISABLE	28
4.2.3.4	MCCL_SOCKET_FAMILY	28
4.2.3.5	MCCL_SOCKET_IFNAME	28
4.2.3.6	MCCL_IB_HCA	28
4.2.3.7	MCCL_NET_GDR_LEVEL	29
4.2.4	调试相关	29
4.2.4.1	MCCL_DEBUG	29
4.2.4.2	MCCL_DEBUG_SUBSYS	30
4.2.4.3	MCCL_DEBUG_FILE	30
4.2.5	UMD 相关	31
4.2.5.1	FORCE_ACTIVE_WAIT	31
4.2.5.2	MACA_LAUNCH_BLOCKING	31
4.2.5.3	MACA_VISIBLE_DEVICES	31
4.2.5.4	MACA_DEVICE_ORDER	32
4.2.5.5	MXLOG_LEVEL	32
4.2.5.6	MACA_LAUNCH_MODE	33
4.2.5.7	MACA_MPS_MODE	33
4.2.5.8	MACA_DIRECT_DISPATCH	33
4.2.6	工具相关	33
4.2.6.1	MX_TRACER_ENABLED_MCPTI	34
4.2.6.2	MCCL_OPTIMIZATION_A2A	34
4.2.6.3	P2P_MODE	34
5	附录	35
5.1	调试信息	35
5.1.1	共享内存	35
5.2	术语/缩略语	35

图目录

图 2.1 系统架构	3
------------------	---

更新记录

版本	日期	更新说明
V06	2025-04-21	更新以下章节： 4 环境变量
V05	2025-02-18	更新以下章节： 2 简介 3.2 集合通讯
V04	2024-06-14	更新以下章节： 3.1 通讯器管理
V03	2024-03-15	新增曦云 [®] 系列 GPU 产品信息 更新以下章节： 4 环境变量
V02	2023-12-29	新增以下章节： 5.1.1 共享内存
V01	2023-10-16	首次发布

1 概述

本文档详细描述了使用曦云[®] 系列 GPU MetaX Collective Communications Library (MCCL) 的软件开发方法，旨在帮助开发人员利用曦云系列 GPU 提供的计算资源，快速构建自己的应用。

本文档主要适用于利用曦云系列 GPU MCCL 的软件开发人员。

2 简介

MetaX Collective Communications Library (MCCL) 是一个提供 GPU 间通信原语的库。MCCL 使用方便，可以进行拓扑感知与计算，是专用于 GPU 间通信的通讯库。

MCCL 提供以下集合通信原语：

- AllReduce
- Broadcast
- Reduce
- AllGather
- ReduceScatter
- AllToAll
- P2P Send/Receive

MCCL 是基于 MXMACA[®]，并通过 MXMACA 的内存复制和 MXMACA 核函数来实现，以进行局部归约。另一方面，MCCL 通过算法优化，以实现快速同步，并最大限度地提高带宽利用率。

开发人员在使用 MCCL 后，不再需要对通讯进行优化。MCCL 支持节点内和跨节点的多 GPU 上进行集合通讯。它支持多种技术，包括 PCIe、MetaXLink、InfiniBand Verbs 和 IP Sockets。

MCCL 编程简单，提供 C 语言标准的 API，可以通过多种编程语言调用。MCCL 符合由 MPI 定义的 API 标准。MCCL 增加了 stream 参数，可以与 MXMACA 进行同步。MCCL 兼容多种 GPU 并行化模型：

- 所有 GPU 的单线程控制
- 多线程，例如，每个 GPU 使用一个线程
- 多进程，例如 MPI

MCCL 在深度学习框架中得到了很好的应用，其中 AllReduce 被大量用于神经网络训练。MCCL 提供的多 GPU 多节点通信可以有效扩展神经网络训练。

2.1 系统架构

曦云系列 GPU MCCL 的系统整体架构如图 2.1 所示：

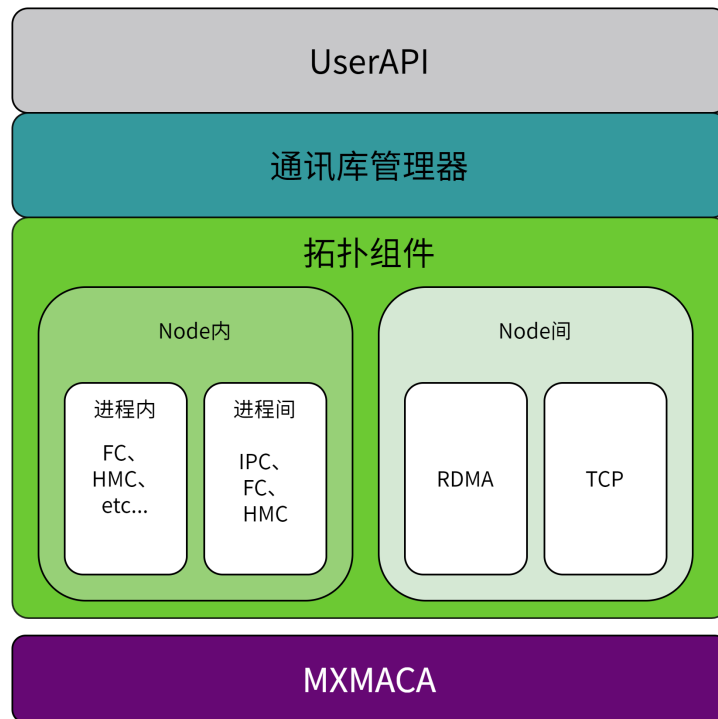


图 2.1 系统架构

3 编程模型

3.1 通讯器管理

3.1.1 mcclGetVersion

```
mcclResult_t mcclGetVersion(int* version);
```

获取当前 MCCL 版本号。

3.1.2 mcclGetUniqueId

```
mcclResult_t mcclGetUniqueId(mcclUniqueId* uniqueId);
```

获取 ID 号，用于在多进程场景中初始化的 ID，由任意一个进程获取，并通过 MPI 或 socket 发送给其它进程用于初始化。多节点环境同样适用。

3.1.3 mcclCommInitRank

```
mcclResult_t mcclCommInitRank(mcclComm_t* comm, int nranks, mcclUniqueId commId,  
    ↪ int rank);
```

创建通讯器，nranks 为通讯器总数，commId 为使用 mcclGetUniqueId 获取到的 ID，rank 为此通讯器的编号，其最小值为 0，最大值为 nranks-1。此通讯器所使用的设备为当前线程正在使用的设备。每个通讯器不能使用相同的设备。

3.1.4 mcclCommInitAll

```
mcclResult_t mcclCommInitAll(mcclComm_t* comm, int ndev, const int* devlist);
```

用于在单进程场景中一次性创建所有通讯器。devlist 包含了每个 rank 使用的设备。

3.1.5 mcclCommDestroy

```
mcclResult_t mcclCommDestroy(mcclComm_t comm);
```

销毁通讯器，结束集合通讯，用于正常通讯流程的退出。

3.1.6 mcclCommAbort

```
mcclResult_t mcclCommAbort(mcclComm_t comm);
```

中止通讯器，结束集合通讯，常用于从致命异常（如网络故障、节点故障或进程故障）中恢复通讯。当异常出现时，调用此 API，然后创建新的通信器重新通讯。

3.1.7 mcclGetErrorString

```
const char* mcclGetErrorString(mcclResult_t result);
```

获取返回值的字符串。

3.1.8 mcclCommGetAsyncError

```
mcclResult_t mcclCommGetAsyncError(mcclComm_t comm, mcclResult_t* asyncError);
```

获取异步操作的返回值。

3.1.9 mcclCommCount

```
mcclResult_t mcclCommCount(const mcclComm_t comm, int* count);
```

获取通讯器数量。

3.1.10 mcclCommMcDevice

```
mcclResult_t mcclCommMcDevice(const mcclComm_t comm, int* device);
```

获取当前通讯器所使用的 device。

3.1.11 mcclCommUserRank

```
mcclResult_t mcclCommUserRank(const mcclComm_t comm, int* rank);
```

获取当前通讯器的 rank 编号。

3.1.12 示例

3.1.12.1 单进程

```
int nranks = 4;
mcclComm_t comms[nranks];
int devs[nranks] = {0, 1, 2, 3};
mcclCommInitAll(comms, nranks, devs);
//...
//OPs...
//...
// finalizing MCCL
for (int i = 0; i < nranks; ++i)
    mcclCommDestroy(comms[i]);
```

3.1.12.2 多进程

```
mcclUniqueId id;
mcclComm_t comm;

//initializing MPI
MPICHECK(MPI_Init(&argc, &argv));
MPICHECK(MPI_Comm_rank(MPI_COMM_WORLD, &myRank));
MPICHECK(MPI_Comm_size(MPI_COMM_WORLD, &nRanks));

//get MCCL unique ID at rank 0 and broadcast it to all others
if (myRank == 0) mcclGetUniqueId(&id);
MPICHECK(MPI_Bcast((void *)&id, sizeof(id), MPI_BYTE, 0, MPI_COMM_WORLD));

mcSetDevice(myRank);
mcclCommInitRank(&comm, nRanks, id, myRank);

//Other Ops.

//finalizing MCCL
mcclCommDestroy(comm);
```

3.2 集合通讯

所有 rank 都必须参与集合通讯，即每一种操作必须被所有 rank 调用，否则其它 rank 将等待。

3.2.1 mcclAllReduce

```
mcclResult_t mcclAllReduce(const void* sendbuff, void* recvbuff, size_t count,
    ↪mcclDataType_t datatype, mcclRedOp_t op, mcclComm_t comm, mcStream_t stream);
```

在所有 rank 上都执行归约计算，最终所有 rank 将得到相同的结果，并保存在 recvbuff 中。sendbuff 为输入源，count 为数据的个数，datatype 为数据类型，op 为操作类型，comm 为通信器对象，stream 为 MXMACA 的 stream。

3.2.2 mcclBroadcast

```
mcclResult_t mcclBroadcast(const void* sendbuff, void* recvbuff, size_t count,
    ↪mcclDataType_t datatype, int root, mcclComm_t comm, mcStream_t stream);
```

```
mcclResult_t mcclBcast(void* buff, size_t count, mcclDataType_t datatype, int root,
    ↪ mcclComm_t comm, mcStream_t stream);
```

mcclBroadcast 可将 sendbuff 中的数据复制到所有 rank 的 recvbuff 中 (包括发送者的 recvbuff)。count 为数据的个数，datatype 为数据类型，root 为发送者的 rank 编号，comm 为通信器对象，stream 为 MXMACA 的 stream，对于接收者，sendbuff 可为空。

mcclBcast 与 mcclBroadcast 相同，若发送者的 sendbuff 与 recvbuff 相同，则可以用 mcclBcast 代替 mcclBroadcast。

3.2.3 mcclReduce

```
mcclResult_t mcclReduce(const void* sendbuff, void* recvbuff, size_t count,
    ↪mcclDataType_t datatype, mcclRedOp_t op, int root, mcclComm_t comm, mcStream_t
    ↪stream);
```

在 rank 为 root 的通信器上执行归约计算，结果保存在 root 的 recvbuff 中。对于其它 rank，recvbuff 可为空。对于发送方，root 为发送的目标 rank。root 为执行归约的通信器 rank 号，comm 为通信器对象，stream 为 MXMACA 的 stream。

3.2.4 mcclAllGather

```
mcclResult_t mcclAllGather(const void* sendbuff, void* recvbuff, size_t sendcount,
    ↪mcclDataType_t datatype, mcclComm_t comm, mcStream_t stream);
```

将所有 rank 的 sendbuff 中的数据按 rank 顺序排列保存到每一个 rank 的 recvbuff 中, 最终每个 rank 得到相同的结果。sendcount 为 sendbuff 的数据个数, recvbuff 的数据个数将会是 sendcount*N (N 为所有 rank 的个数)。datatype 为数据类型, comm 为通信器对象, stream 为 MXMACA 的 stream。调用者必须确保 recvbuff 有足够的空间。

3.2.5 mcclReduceScatter

```
mcclResult_t mcclReduceScatter(const void* sendbuff, void* recvbuff, size_t
    ↪recvcount, mcclDataType_t datatype, mcclRedOp_t op, mcclComm_t comm, mcStream_t
    ↪stream);
```

对所有 rank 的 sendbuff 进行归约, 然后将结果分成 N 份, 并按 rank 顺序分发给所有通信器的 recvbuff 中。recvcount 为接收数据的个数, 发送数据的个数是 recvcount*N。op 为操作类型, comm 为通信器对象, stream 为 MXMACA 的 stream。

3.2.6 mcclAllToAll

```
mcclResult_t mcclAllToAll(const void* sendbuff, void* recvbuff, size_t count,
    ↪mcclDataType_t datatype, mcclComm_t comm, mcStream_t stream);
```

在所有 rank 上执行全交换操作, 每个 rank 将 sendbuff 中的数据发送给所有其他 rank, 并从其他 rank 接收数据到 recvbuff 中。每个 rank 发送和接收的数据量相同, 均为 count 个 datatype 类型的数据。最终, 每个 rank 的 recvbuff 中将包含来自所有 rank 的数据, 且数据顺序与 rank 编号一致。sendbuff 为发送数据的缓冲区, recvbuff 为接收数据的缓冲区, count 为每个 rank 发送和接收的数据个数。datatype 为数据类型, comm 为通信器对象, stream 为 MXMACA 的 stream。调用者必须确保 recvbuff 有足够的空间来存储所有 rank 发送的数据。

3.2.7 mcclAllToAllv

```
mcclResult_t mcclAllToAllv(const void* sendbuff, const size_t sendcounts[], const
    ↪size_t sdispls[], void* recvbuff, const size_t recvcounts[], const size_t
    ↪rdispls[], mcclDataType_t datatype, mcclComm_t comm, mcStream_t stream);
```

在所有 rank 上执行非均匀全交换操作, 每个 rank 可以发送不同数量的数据给其他 rank, 并从其他 rank 接收不同数量的数据。sendbuff 为发送数据的缓冲区, sendcounts 为数组, sendcounts[i] 表示发送给 rank i 的数据个数。sdispls 为数组, sdispls[i] 表示发送给 rank i 的数据在 sendbuff 中的起始偏移量。recvbuff 为接收数据的缓冲区, recvcounts 为数组, recvcounts[i] 表示从 rank i 接收的数据个数。rdispls 为数组, rdispls[i] 表示从 rank i 接收的数据在 recvbuff 中的起始

偏移量。datatype 为数据类型，comm 为通信器对象，stream 为 MXMACA 的 stream。调用者必须确保 sendbuff 和 recvbuff 有足够的空间，且 sendcounts、sdispls、recvcounts 和 rdispls 的数组长度必须与通信器中的 rank 数量一致。

3.2.8 mcclAllToAlld

```
mcclResult_t mcclAllToAlld(const void* sendbuff[], const size_t sendcounts[],
    ↪void* recvbuff[], const size_t recvcounts[], mcclDataType_t datatype, mcclComm_t
    ↪comm, mcStream_t stream);
```

在所有 rank 上执行分布式全交换操作，每个 rank 发送多个数据块给其他 rank，并从其他 rank 接收多个数据块。sendbuff 和 recvbuff 是数组的数组，分别表示每个 rank 发送和接收的数据块。sendcounts 和 recvcounts 分别指定每个 rank 发送和接收的数据块的大小。sendbuff 为数组，sendbuff[i] 指向发送给 rank i 的数据缓冲区，sendcounts 为数组，sendcounts[i] 表示发送给 rank i 的数据个数。recvbuff 为数组，recvbuff[i] 指向接收来自 rank i 的数据缓冲区，recvcounts 为数组，recvcounts[i] 表示从 rank i 接收的数据个数。datatype 为数据类型，comm 为通信器对象，stream 为 MXMACA 的 stream。调用者必须确保 sendbuff 和 recvbuff 中的每个缓冲区有足够的空间，且 sendbuff、recvbuff、sendcounts 和 recvcounts 的数组长度必须与通信器中的 rank 数量一致。

3.2.9 示例

```
mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclAllReduce((const void*)sendbuff[i], (void*)recvbuff[i], size, mcclFloat,
    ↪mcclSum, comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclBroadcast((const void*)sendbuff[i], (void*)recvbuff[i], size, mcclFloat, 0,
    ↪comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclReduce((const void*)sendbuff[i], (void*)recvbuff[i], size, mcclFloat,
    ↪mcclSum, 0, comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
```

(下页继续)

(续上页)

```

    mcclAllGather((const void*)sendbuff[i], (void*)recvbuff[i], size, mcclFloat,
    ↪comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclAllReduceScatter((const void*)sendbuff[i], (void*)recvbuff[i], size,
    ↪mcclFloat, mcclSum, comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclAllToAll((const void*)sendbuff[i], (void*)recvbuff[i], size, mcclFloat,
    ↪comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclToAllAllv((const void*)sendbuff[i], sendcounts, sdispls, (void*)recvbuff[i],
    ↪recvcounts, rdispls, mcclFloat, comms[i], s[i]);
}
mcclGroupEnd();

mcclGroupStart();
for (int i = 0; i < nranks; ++i) {
    mcclAllToAllld((const void**)sendbuff[i], sendcounts, (void**)recvbuff[i],
    ↪recvcounts, mcclFloat, comms[i], s[i]);
}
mcclGroupEnd();

```

3.3 组调用

组调用可以把一系列 MCCL 操作合并在一起放入后台执行，不同 stream 的操作可互不影响地执行。这样可以实现 MCCL 操作在不同 stream 并发执行。

3.3.1 mcclGroupStart

```
mcclResult_t mcclGroupStart();
```

开始一个组编程。在此之后的所有 MCCL 函数都不会立即执行，而是放入一个执行队列中等待执行。

3.3.2 mcclGroupEnd

```
mcclResult_t mcclGroupEnd();
```

结束组编程。立即启动队列中的所有操作，并根据不同 stream 放入不同队列执行。在所有操作完成之前，阻塞当前线程。

3.3.3 示例

```
mcclGroupStart();
mcclSend(sendbuff[0], size, mcclFloat, 1, comms[0], s[0]);
mcclRecv(recvbuff[1], size, mcclFloat, 0, comms[1], s[1]);
mcclGroupEnd();
```

3.4 点对点通讯

点对点的通讯，即把数据从一个 rank 复制给另一个 rank，也就是将显存中的内容，由一个显卡复制到另一个显卡中。

3.4.1 mcclSend

```
mcclResult_t mcclSend(const void* sendbuff, size_t count, mcclDataType_t datatype,
    ↪ int peer, mcclComm_t comm, mcStream_t stream);
```

将 sendbuff 中的内存发送给接收端 peer。count 为数据的个数，datatype 为数据类型。

comm 为通信器对象，stream 为 MXMACA 的 stream。

3.4.2 mcclRecv

```
mcclResult_t mcclRecv(void* recvbuff, size_t count, mcclDataType_t datatype, int  
↪peer, mcclComm_t comm, mcStream_t stream);
```

从 peer 端接收数据到 recvbuff 中。count 为数据的个数，datatype 为数据类型。

comm 为通信器对象，stream 为 MXMACA 的 stream。

3.4.3 示例

```
mcclSend(sendbuff[0], size, mcclFloat, 1, comms[0], s[0]);  
mcclRecv(recvbuff[1], size, mcclFloat, 0, comms[1], s[1]);
```

3.5 数据类型

MCCL 的数据类型用于指定各操作原语所使用的数据类型，对应 MXMACA 的数据类型。对应关系如下：

- mcclInt8: char
- mcclChar: char
- mcclUInt8: unsigned char
- mcclInt32: int
- mcclInt: int
- mcclUInt32: unsigned int
- mcclInt64: long
- mcclUInt64: unsigned long
- mcclFloat16: half
- mcclHalf: half
- mcclFloat32: float
- mcclFloat: float
- mcclFloat64: double
- mcclDouble: double
- mcclBfloat16: bfloat16

3.6 返回值类型

- `mcclSuccess`: 成功
- `mcclUnhandledMacaError`: 调用 MXMACA API 时返回错误
- `mcclSystemError`: 调用系统函数错误
- `mcclInternalError`: MCCL 内部错误, 可能是内存错误
- `mcclInvalidArgument`: 参数错误
- `mcclInvalidUsage`: 使用错误, 可能是用户编程导致
- `mcclRemoteError`: 多节点场景时, 网络连接错误

3.7 归约操作类型

- `mcclSum`: 求和
- `mcclProd`: 求乘积
- `mcclMax`: 求最大值
- `mcclMin`: 求最小值
- `mcclAvg`: 求平均值

4 环境变量

4.1 推荐环境变量

常规情况下，使用 MCCL 不需要设置环境变量。如曦云 C500/C550 等 x86 机型。

针对下述场景，需要单独设置环境变量来满足特定需求。

4.1.1 多机环境

集群多机网络如果是常规配置，通常不需要配置网络相关的环境变量。为了屏蔽网络设置差异，可以设置如下环境变量简化配置，保障多机正常运行。

```
export MCCL_IB_HCA=mlx5_0,mlx5_1 (可用的计算网卡网口，不包括存储网卡网口)
export MCCL_NET_GDR_LEVEL=SYS
export MCCL_CROSS_NIC=1
```

4.1.2 极致性能

可设置如下环境变量提升通信性能，尤其是运行 MCCL 性能测试时。部分环境变量会对其他性能产生一些影响，在大模型中使用需要权衡。

- UMD 环境变量，降低通信的 CPU 时延，但会增加 CPU 负载：

```
export FORCE_ACTIVE_WAIT=2
```

- 提升 MCCL 使用 Cache 的性能，但可能会对 BLAS Kernel 产生性能影响：

```
export MCCL_FAST_WRITE_BACK=1
```

- 提升 MCCL 使用 Cache 的性能，但可能会对 BLAS Kernel 产生性能影响：

```
export MCCL_EARLY_WRITE_BACK=15
```

- 开启 PCIe 链路。OAM 机型上默认关闭 PCIe 链路，测试 bench perf 可以开启以提升性能：

```
export MCCL_PCIE_BUFFER_MODE=1
```

- 开启 MCCL 性能测试中的 All to All 性能优化：

```
export MCCL_OPTIMIZATION_A2A=1
```

- UMD 环境变量，可降低 GPU 计算时延，取消 GPU 任务完成后不必要的刷新 L2 缓存行为，对小尺寸内核（small size kernel）的时延优化约 1 us：

```
export MACA_LAUNCH_MODE=1
```

4.1.3 Arm 服务器

- Cascade 拓扑（天固服务器）：

```
export MACA_VISIBLE_DEVICES=0,1,6,7,2,3,4,5
```

- Balance topo 单机内使用网卡做 8 卡通信：

```
export MCCL_SHM_DISABLE=1
```

- 通过 NUMA 节点绑定核心的方式来提高性能，需要以下变量：

```
export MCCL_IGNORE_CPU_AFFINITY=1
```

4.1.4 异构集群

- 沐曦和其他厂商 GPU 构建异构集群：

```
export MCCL_EXT_CCL_ENABLE=1
```

- 指定异构插件：

```
export MCCL_HC_PLUGIN=${your_plugin_dynamic_library_path}
```

4.1.5 虚拟化场景

- 关闭 PCIe。开启虚拟机需要开启 ACS，会导致 PCIe P2P 性能变差：

```
export MCCL_PCIE_BUFFER_MODE=0
```

- 强制 UMD 使用 polling 模式，降低小数据量的通信时延。CPU 性能强的情况下，可以设置为 3：

```
export FORCE_ACTIVE_WAIT=2
```

- 来指定特定拓扑信息：

```
export MCCL_TOPO_FILE=path/topo.xml
```

4.1.6 C500X 机型

软件方式开关 C500X Dragonfly 机型的机外互联，可使用环境变量 `MCCL_DISABLE_OPTIC_LINK`，详情参见 4.2.1.1 `MCCL_DISABLE_OPTIC_LINK`。

4.1.7 阡视服务器

针对阡视 8 卡性能最优服务器，需要设置：

```
MACA_VISIBLE_DEVICES=0,1,8,9,2,3,10,11,4,5,12,13,6,7,14,15。
```

4.1.8 大模型通信日志

获取大模型的通信行为日志，需要设置：

```
export MCCL_DEBUG=TRACE
export MCCL_DEBUG_SUBSYS=^ALLOC
```

4.2 环境变量说明

4.2.1 功能相关

4.2.1.1 `MCCL_DISABLE_OPTIC_LINK`

描述：C500X 环境使用软件方式关闭机外 MetaXLink 互联的环境变量。在机外互联 training 成功后使用，通信效果和不 training 一致。

可选值：

- 0：开启机外互联
- 1：关闭机外互联

默认值：0

4.2.1.2 `MCCL_FAST_WRITE_BACK`

描述：使用 MCCL 内存屏障（Memory Barrier）时是否启用快速回写 Cache Line 的功能。

可选值：

- -2：不启用
- 其他值：启用

默认值：-2

4.2.1.3 MCCL_EARLY_WRITE_BACK

描述：L2C 回写阈值设置，超过阈值时才进行回写。

可选值：

- -2：不启用回写
- 其他值：数据超过此值时进行回写

默认值：-2

4.2.1.4 MCCL_GROUP_WRITE_BACK

描述：是否启用 Channel 分组功能，同组的 Channel 只做一次 Memory Barrier。

可选值：

- 0：启用
- 1：不启用

默认值：1

4.2.1.5 MCCL_DISABLE_MULTI_NODE_FABRIC

描述：是否使用 Dragonfly 两机间的光链路。

可选值：

- 0：使用
- 1：不使用

默认值：0

4.2.1.6 MCCL_BUFFSIZE

描述：控制 MCCL 在 GPU 对之间传输数据时使用的缓冲区大小（对应 simple 协议）。如果在使用 MCCL 时遇到内存限制问题，或者认为不同的缓冲区大小可以提高性能，请使用此变量。

可选值：使用整数值，建议使用 2 的幂

默认值：8388608（8 MB）

4.2.1.7 MCCL_DISABLE_CACHEABLE_BUFFER

描述：是否支持使用 RWK Buffer。

可选值：

- 0：不使用 RWK Buffer
- 1：使用 RWK Buffer

默认值：0

4.2.1.8 MCCL_THRESHOLD_TO_USE_CACHEABLE_BUFFER

描述：simple 模式下使用 RWK Buffer 的阈值，大于此值时使用 RWK Buffer

可选值：

- -1：使用默认阈值，normal node 模式下为 134217728 (128M)，dragonfly node 模式下为 536870912 (512M)
- 其他值：使用整数值，建议使用 2 的幂

默认值：-1

4.2.1.9 MCCL_THRESHOLD_TO_USE_CACHEABLE_BUFFER_P2P

描述：simple p2p 模式下使用 RWK Buffer 的阈值，大于此值时使用 RWK Buffer。

可选值：

- -1：使用默认阈值 134217728 (128M)
- 其他值：使用整数值，建议使用 2 的幂

默认值：-1

4.2.1.10 MCCL_PCIE_BUFFER_MODE

描述：设置 MCCL 中 PCIe Buffer 的工作模式，用于选择卡间通信是否使用 PCIe 链路进行通信。

可选值：

- -1：自动选择，由服务器的 GPU 拓扑决定
- 0：首选 MetaXLink
- 1：选择 MetaXLink 和 PCIe
- 2：只选择 PCIe

默认值：-1

4.2.1.11 MCCL_TUNING_MODEL

描述：根据全局拓扑的种类设置了不同的算法选择适配模式，不同值会使用不同的算法固定延迟、带宽系数以及对应不同算法不同数据大小的带宽微调。

可选值：

- 5：单节点时推荐设置为 5
- 6：多节点时推荐设置为 6

默认值：5

4.2.1.12 MCCL_PROTO

描述：定义 MCCL 将使用哪种协议。不建议用户设置此变量，除非在怀疑 MCCL 中存在错误的情况下禁用特定协议。特别是在不支持 LL128 的平台上启用 LL128 可能会导致数据损坏。

可选值：以逗号分隔的协议列表（不区分大小写），包括：LL、LL128、Simple。要指定要排除（而不是包含）的协议，请以 ^ 开头列表。

默认值：在支持 LL128、LL、Simple 的平台上，默认值为 `LL,LL128,Simple`。

4.2.1.13 MCCL_ALGO

描述：定义 MCCL 将使用哪些算法。

可选值：以逗号分隔的算法列表（不区分大小写），包括：树、环、Collnet、CollnetDirect 和 CollnetChain。要指定要排除（而不是包含）的算法，请以 ^ 开始列表。

默认值：Tree,Ring,CollnetDirect,CollnetChain

4.2.1.14 MCCL_DMABUF_ENABLE

描述：使用 Linux dma-buf 子系统启用 GPU Direct RDMA 缓冲区注册。Linux dma-buf 子系统允许支持 GPU Direct RDMA 的 NIC 直接读取和写入 MXMACA 缓冲区，而无需 CPU 参与。

可选值：

- 0：禁用
- 1：启用

默认值：1

如果 Linux 内核或 MXMACA/NIC 驱动程序不支持该功能，则会自动禁用该功能。

4.2.1.15 MCCL_EXT_CCL_ENABLE

描述：启用混训功能。

可选值：

- 0：禁用
- 1：启用

默认值：0

4.2.1.16 MCCL_HC_PLUGIN

描述：指定异构环境使用的特定版本插件动态库。通过配置该变量，可以将沐曦 GPU 与指定其它厂商的 GPU 进行异构通信。

可选值：插件动态库路径

默认值：/opt/maca/libmxccl.so

4.2.1.17 MCCL_RINGS

描述：用户自定义环路，只在固定序号的 GPU 上进行 ring 运算。

可选值：

字符串格式为 0 1|1 0|0 1 2 3|3 2 1 0|N0 0 2 3 1|N2 7 6 5 4 3 2 1 0 N1

其中网卡可选择指定字符前缀 N 表示。PCIe P2P 可以指定字符前缀 P 表示。

默认值：无

4.2.2 性能相关

4.2.2.1 MCCL_ENABLE_FC

描述：是否启用 FC 算法。

可选值：

- 0：关闭
- 1：启用

默认值：1

4.2.2.2 MCCL_ENABLE_FC8_OAM

描述：在 8 卡 OAM 拓扑下启用 FC 算法。

可选值：

- 0：关闭
- 1：启用

默认值：1

4.2.2.3 MCCL_FC_BYTE_LIMIT

描述：使用 FC 算法的数据量上限。

可选值：无符号长整形数值

默认值：4294967296

4.2.2.4 MCCL_FC_MAX_BLOCKS

描述：限制 FC 算法使用的 block 数量。

可选值：正整数值，其中 0 表示由算法自动选择

默认值：32

4.2.2.5 MCCL_FC_DISABLE_REMOTE_READ

描述：关闭 FC 远读算法。

可选值：

- 1：关闭
- 0：打开

默认值：1

4.2.2.6 MCCL_FC_BYTE_LIMIT_DRAGONFLY

描述：限制 Dragonfly 拓扑下使用 FC 分层算法的数据量上限。

可选值：无符号长整型数值

默认值：2097152

4.2.2.7 MCCL_FC_MTLK_BLOCKS

描述：限制 FC8 分层算法的 MetaXLink 部分 block 数量。

可选值：无符号长整型数值

默认值：0，由算法自动选择

4.2.2.8 MCCL_LIMIT_RING_LL_THREADTHRESHOLDS

描述：在 Ring 算法 LL 协议场景下是否限制。

可选值：

- 0：关闭
- 1：启用

默认值：1

4.2.2.9 MCCL_CROSS_NIC

描述：控制 MCCL 是否允许 ring/trees 使用不同的网卡，导致节点间通信在不同节点上使用不同的网卡。

为了在使用多个网卡时最大限度地提高节点间的通信性能，MCCL 在节点间通信时尽量使用相同的网卡，允许每个节点上的每个网卡连接到不同的网络交换机（网络轨道）的网络设计，避免任何流量干扰的风险。因此，MCCL_CROSS_NIC 设置取决于网络拓扑，特别是取决于网络结构是否经过轨道优化。

这对只有一个网卡的系统没有影响。

可选值：

- 0：始终在同一个环/树中使用相同的 NIC（网络接口卡），以避免跨越网络轨道。这适用于每个 NIC 都连接到独立交换机（轨道）、且轨道间连接较慢的网络。请注意，如果通信器中每个节点上的 GPU 并不完全相同，MCCL 可能仍需要跨 NIC 进行通信。
- 1：允许在同一个环/树中使用不同的 NIC。这适用于所有节点的 NIC 都连接到同一个交换机的网络，因此尝试仅使用相同的 NIC 并不能避免流量冲突。
- 2：尽量在同一个环/树中使用相同的 NIC，但在能够获得更好性能的情况下，也允许使用不同的 NIC。

默认值：2

4.2.2.10 MCCL_MIN_NCHANNELS

描述：限制 MCCL 使用的最小 Channel 数量。增加 Channel 数量会增加 MCCL 使用的 block 数量，这会提升性能但会使用更多的 GPU 计算资源。在一些 MCCL 经常只创建一个 Channel 的平台使用聚合集合通信时，增加 Channel 数量通常会带来性能提升。

注解：旧的变量 `MCCL_MIN_NRINGS` 仍然可以作为别名使用。如果设置了 `MCCL_MIN_NCHANNELS`，将覆盖 `MCCL_MIN_NRINGS`。

可选值：整数值。当 Channel 数为-2 时，最终结果会配置为默认值。当 Channel 数小于 0 且不等于-2 时，会配置为 0。当 Channel 数大于 `MAXCHANNELS`（同构拓扑 `MAXCHANNELS=64`，异构 `MAXCHANNELS=32`）时，会配置为 `MAXCHANNELS`。

默认值：2

4.2.2.11 MCCL_MAX_NCHANNELS

描述：限定了 MCCL 可以使用的 Channel 数量。减少 Channel 数量会减少通信库使用的 block 数量，相应的会影响 GPU 的计算资源。

注解：旧的 `MCCL_MAX_NRINGS` 仍然可以作为别名使用。如果设置了 `MCCL_MAX_NCHANNELS`，将覆盖 `MCCL_MAX_NRINGS`。

可选值：大于等于 1 的整数值。当 Channel 数为-2 时，最终结果会配置为默认值。当 Channel 数小于 0 且不等于-2 时，会配置为 0。当 Channel 数大于 `MAXCHANNELS`（同构拓扑 `MAXCHANNELS=64`，异构 `MAXCHANNELS=32`）时，会配置为 `MAXCHANNELS`。当 `MCCL_MIN_NCHANNELS` 设置的值比 `MCCL_MAX_NCHANNELS` 设置的大，最终结果会取两者最大值。

默认值：32

4.2.2.12 MCCL_RING_TP8_MODE

描述：设定当前为 Dragonfly TP8 拓扑模式，并指定预设的通信环路。

可选值：

- 0：不适用预设通信环路
- 1：使用 8 opt 环路
- 2：使用 8 opt 环路和 3 PCIe 环路
- 3：使用 16 opt 和 6 RC 和 8 RoCE 环路
- 4：使用 16 opt 环路和 6 RC 环路

默认值：0

4.2.2.13 MCCL_NET_DISABLE_INTRA

描述：Intra-node 通信时，当使用网卡通信速度高于 P2P 或 SHM 时，允许优先使用网卡。

可选值：

- 0：允许优先使用网卡
- 1：不允许优先使用网卡

默认值：1

4.2.2.14 MCCL_PXN_DISABLE

描述：禁止节点内通信使用 PXN，即通过 MetaXLink、中间 GPU 和非 local 网卡进行通信。

可选值：

- 0：非禁止
- 1：禁止

默认值：0

4.2.2.15 MCCL_MIN_P2P_NCHANNELS

描述：MCCL 可用在 P2P 通信的最小 Channel 数量。

可选值：整数值

默认值：1

4.2.2.16 MCCL_MAX_P2P_NCHANNELS

描述：MCCL 可用在 P2P 通信的最大 Channel 数量。

可选值：整数值

默认值：64

4.2.2.17 MCCL_P2P_NCHANNELS

描述：设置了 MCCL 在 P2P 通信时使用的 Channel 数量。

可选值：整数值

默认值：12

4.2.2.18 MCCL_TUNING_FILE

描述：用户指定调优配置文件路径。

可选值：可访问的用户自定义文件路径

默认值：/opt/maca/etc/tuning.cfg

4.2.2.19 MCCL_TOPO_FILE

描述：在检测拓扑之前要加载的 XML 文件路径。默认情况下，MCCL 将加载 **/var/run/metax/topo.xml** (如果存在)。

可选值：描述部分或全部拓扑的可访问文件的路径

默认值：/var/run/metax/topo.xml

4.2.2.20 MCCL_TOPO_DUMP_FILE

描述：检测后要将 XML 格式拓扑存储的文件路径。

可选值：要创建或者覆盖的文件路径

默认值：同 **MCCL_TOPO_FILE**

4.2.2.21 MCCL_P2P_DISABLE

描述：禁用基于 PCIe 或 MetaXLink 的点对点（P2P）传输。

可选值：

- 1：禁用
- 0：不禁用

默认值：0

4.2.2.22 MCCL_P2P_LEVEL

描述：允许用户精细地控制何时在 GPU 之间使用点对点传输。该变量定义了 MCCL 将使用 P2P 传输的 GPU 之间的最大距离。应该使用表示路径类型的短字符串来指定使用 P2P 传输的地形截止点。如果没有指定，MCCL 将尝试根据运行的体系结构和环境选择最佳值。

可选值：

- 字符串类型值：
 - LOC：永远不要使用 P2P（总是禁用）
 - MetaXLink：GPU 通过 MetaXLink 连接时使用 P2P
 - PIX：GPU 位于同一 PCI 交换机时，使用 P2P

- PXB: 当 GPU 通过 PCI 交换机（可能有多跳）连接时，使用 P2P
- PHB: GPU 位于同一个 NUMA 节点时使用 P2P，流量将通过 CPU
- SYS: 在 NUMA 节点之间使用 P2P，可能会跨越 SMP 互连（例如 QPI/UPI）
- 整数类型值（Legacy）

可以选择将 `MCCL_P2P_LEVEL` 声明为与路径类型对应的整数。对于那些在允许字符串之前使用数值的人来说，保留这些数值是为了向后兼容。

不建议使用整数值，因为路径类型后续可能更改，数值与类型字符串标识符的映射关系可能会相应变化。为了避免调试配置时遇到的麻烦，请使用字符串标识符。

- LOC: 0
- PIX: 1
- PXB: 2
- PHB: 3
- SYS: 4，大于 4 的值将被解释为 SYS。MetaXLink 不支持使用旧的整数值

默认值：无

4.2.2.23 MCCL_DF16_RINGS

描述：Dragonfly 16 卡拓扑使用的基础环路路径。

可选值：通过 GPU 的 device ID 构建的环路路径

默认值：

```
0 4 7 3 6 2 1 5 15 11 8 12 9 13 14 10|3 7 4 0 5 1 2 6 12 8 11 15 10 14 13 9|4 3 0 7 2 5 6 1 11 12 15 8 13 10
9 14|7 0 3 4 1 6 5 2 8 15 12 11 14 9 10 13|10 14 13 9 8 11 15 5 1 2 6 3 7 4 0|9 13 14 10 15 11 8 12 6 2 1 5 0
7 3|14 9 10 13 8 15 12 11 1 6 5 2 7 0 3 4|13 10 9 14 11 12 15 8 2 5 6 1 4 3 0 7
```

4.2.2.24 MCCL_IGNORE_CPU_AFFINITY

描述：用于让 MCCL 忽略作业提供的 CPU 亲和性，而仅使用 GPU 亲和性。

可选值：

- 0：不忽略 CPU 亲和性
- 1：忽略作业提供的 CPU 亲和性

默认值：0

4.2.2.25 MCCL_RUNTIME_CONNECT

描述：控制是否分配所有算法对应的 Buffer 空间。从性能角度看，数据传输过程中不使用的算法可以不分配 Buffer 空间，以减少内存空间占用。

可选值：

- 0：分配所有算法 Buffer 空间
- 1：只分配所使用算法的 Buffer 空间

默认值：1

4.2.2.26 MCCL_HFC_EP4_KERNEL_LIMIT

描述：控制 AlltoAll 4 卡算法在 Dragonfly Switch 拓扑下的算法选择。该值为 4 卡（每个机器一张卡）使用 HFC 的 kernel 限制值。小于等于该值时使用 ep4 的 kernel。大于该值时使用 send/recv 算法。

可选值：0-0x7fffffffffffffff

默认值：524288

4.2.2.27 MCCL_HFC_EP8_KERNEL_LIMIT

描述：控制 AlltoAll 8 卡算法在 Dragonfly Switch 拓扑下的算法选择。该值为 8 卡（每个机器一张卡）使用 HFC 的 kernel 限制值。小于等于该值时使用 ep8 的 kernel。大于该值时使用 send/recv 算法。

可选值：0-0x7fffffffffffffff

默认值：524288

4.2.3 网络相关

4.2.3.1 MCCL_SHM_DISABLE

描述：设置为 1 后将禁用共享内存（SHM）传输，MCCL 将使用网络（即 InfiniBand 或 IP 套接字）在 CPU 套接字之间进行通信。

可选值：

- 0：启用共享内存（SHM）传输
- 1：关闭共享内存（SHM）传输

默认值：0

4.2.3.2 MCCL_IB_GID_INDEX

描述：定义在 RoCE 模式中 Global ID 索引。

可选值：-1, 0, 正整数

默认值：-1

4.2.3.3 MCCL_IB_DISABLE

描述：可防止 MCCL 使用 IB/RoCE 传输。相反，MCCL 将恢复使用 IP 套接字。

可选值：

- 0：启用 IB/RoCE 传输
- 1：启用 IP 套接字传输

默认值：0

4.2.3.4 MCCL_SOCKET_FAMILY

描述：允许用户强制 MCCL 只使用 IPv4 或 IPv6 接口。

可选值：设为 AF_INET 则强制使用 IPv4。设为 AF_INET6 则强制使用 IPv6。

默认值：无

4.2.3.5 MCCL_SOCKET_IFNAME

描述：指定使用哪些 IP 接口进行通信。

可选值：

定义为前缀列表，用于筛选 MCCL 将使用的接口。可以提供多个前缀，以 , 符号分隔。使用 ^ 符号，MCCL 将排除以该列表中任何前缀开头的接口。要匹配（或不匹配）精确的接口名称，请在前缀字符串的开头使用 = 字符。示例：MCCL_SOCKET_IFNAME=eth0,eth1。

默认值：无

4.2.3.6 MCCL_IB_HCA

描述：指定使用哪些 RDMA 接口进行通信。

可选值：

定义用于筛选 MCCL 将使用的 IB Verbs 接口。列表以 , 分隔；端口号可使用 : 符号指定。可选前缀 ^ 表示该列表是排除列表。第二个可选前缀 = 表示标记为精确名称，否则 MCCL 默认会将每个标记视为接口前缀。示例：MCCL_IB_HCA=mlx5_0,mlx5_1。

默认值：无

4.2.3.7 MCCL_NET_GDR_LEVEL

描述：允许用户精细控制何时在 NIC 和 GPU 之间使用 GPU Direct RDMA。该变量使用表示路径类型的字符串来指定 NIC 和 GPU 之间允许使用 GPU Direct RDMA 的最大距离级别。如果未指定，MCCL 将尝试根据其运行的体系结构和环境以最佳方式选择一个值。

可选值：

- 字符串类型值：
 - LOC：从不使用 GPU Direct RDMA（始终禁用）
 - MetaXLink：GPU 通过 MetaXLink 连接 NIC 时使用 GPU Direct RDMA
 - PIX：当 GPU 和 NIC 位于同一 PCI 交换机上时，使用 GPU Direct RDMA
 - PXB：当 GPU 和 NIC 通过 PCI 交换机（可能是多跳）连接时，使用 GPU Direct RDMA
 - PHB：当 GPU 和 NIC 位于同一 NUMA 节点上时，使用 GPU Direct RDMA，流量将通过 CPU
 - SYS：即使在 NUMA 节点之间的 SMP 互连中也可以使用 GPU Direct RDMA（始终启用）

- 整数类型值（Legacy）

可以选择将 `MCCL_NET_GDR_LEVEL` 声明为对应于路径类型的整数。保留这些数值是为了追溯兼容性，适用于在允许字符串之前使用数值的用户。

不建议使用整数值，因为路径类型后续可能更改，数值与类型字符串标识符的映射关系可能会相应变化。为避免调试配置时遇到麻烦，请使用字符串标识符。

- LOC: 0
- PIX: 1
- PXB: 2
- PHB: 3
- SYS: 4，大于 4 的值将被解释为 SYS

默认值：无

4.2.4 调试相关

4.2.4.1 MCCL_DEBUG

描述：控制 MCCL 日志打印级别，日志级别高的等级包含低等级日志信息。

可选值：

- NONE：日志级别为 0，不打印任何日志
- VERSION：日志级别为 1，打印 MCCL 库版本号信息
- WARN：日志级别为 2，打印 MCCL 错误信息
- INFO：日志级别为 3，打印调试信息

- ABORT: 日志级别为 4, 打印错误终止信息
- TRACE: 日志级别为 5, 打印函数的调用日志信息

默认值: 0, 不打印任何日志信息

4.2.4.2 MCCL_DEBUG_SUBSYS

描述: 用于过滤日志输出的模块, 通过逗号分割, 可以输出多个模块的日志内容。

主要包括: INIT,COLL,P2P,SHM,NET,GRAPH,TUNING,ENV,ALLOC,CALL,DATA

可选值:

- INIT: 初始化模块
- COLL: 集合操作模块
- P2P: Peer To Peer 传输模块
- SHM: 共享内存传输模块
- NET: 网络传输模块
- GRAPH: 拓扑检测和拓扑图搜索
- TUNING: (TREE/RING/COLL) 算法和 (Simple/LL/LL128) 协议调优
- ENV: 环境变量
- ALLOC: 内存申请
- CALL: 函数调用相关信息
- DATA: 传输数据信息打印
- ALL: 所有模块

默认值: 0, 不打印任何日志信息

4.2.4.3 MCCL_DEBUG_FILE

描述: 指定调试日志生成的日志文件, 格式: `filename.%h.%p`, %h 为主机名, %p 为进程号。不支持 ~ 字符, 需要使用相对路径或绝对路径。

可选值: 用户自定义输入

默认值: 为空, 生成的日志文件路径为 **\$HOME/mxlog/mccl.pid.timestamp.log**

4.2.5 UMD 相关

4.2.5.1 FORCE_ACTIVE_WAIT

描述：指定 UMD 中 Host 侧等待事件或流之间同步的 CPU 行为。

可选值：

- 0：默认标准模式，阻塞等待计算任务完成
- 1：该进程抢占 CPU 进行等待相对温和
- 2：该进程抢占 CPU 进行等待比较激进
- 3：该进程抢占 CPU 进行等待最激进
- -1：该进程等待时尽量让出 CPU，可以有效降低在使用大量流场景下的 CPU 负载

默认值：0

4.2.5.2 MACA_LAUNCH_BLOCKING

描述：Host 和 Device 是并发执行的，在同一时间上各自完成不同的任务，许多操作在 Host 和 Device 之间异步完成。该变量强制消除异步行为，强制同步执行，有助于分析和定位问题。

可选值：

- 0：异步并发执行 kernel（计算任务）
- 1：强制同步执行 kernel
- 2：强制同步执行 kernel，kernel 之间增加 sleep 方便区分不同的 kernel

默认值：0

4.2.5.3 MACA_VISIBLE_DEVICES

描述：服务器中有多个 GPU，可以选择特定 GPU 对应用程序的可见性及运行顺序。可以指定 GPU UUID，GPU 设备节点 ID 或者 GPU 的 socket ID（socket ID 当前仅限 Dragonfly 服务器），例如：

- 指定 GPU UUID：

```
export MACA_VISIBLE_DEVICES=GPU-ad23670d-a40e-6b86-6fc3-c44a2cc92c7e
```

- 指定 GPU 设备节点 ID：

```
export MACA_VISIBLE_DEVICES=0,2
```

- 指定 GPU 的 socket id，前缀为 S 或 s：

```
export MACA_VISIBLE_DEVICES=S0,S2,S5
```

或者

```
export MACA_VISIBLE_DEVICES=s0,s2,s5
```

可选值：

UUID/节点 ID/socket ID

- 通过 `mx-smi -L` 命令获取所有 GPU 的 UUID
- 通过 `mx-smi` 命令获取 GPU 节点 ID
- 通过 `grep -rn mgpu_id /sys/class/mxcd/mxcd/layout/nodes` 命令查看 GPU socket ID

默认值：空

4.2.5.4 MACA_DEVICE_ORDER

描述：服务器中有多个 GPU，在程序调用过程中可以按照期望的规则生成 GPU index，C500 支持自定义的多卡排序规则。

可选值：

- FASTEST_FIRST：根据设备计算能力从快到慢排序
- PCI_BUS_ID：根据 PCI 总线 ID 升序排列设备

默认值：FASTEST_FIRST

4.2.5.5 MXLOG_LEVEL

描述：设置 MXMACA UMD 的输出日志等级。

可选值：

- verbose：日志级别为 0，打印 verbose 日志
- debug：日志级别为 1，打印 debug 日志
- info：日志级别为 2，打印 info 日志
- warn：日志级别为 3，打印 warning 日志
- err：日志级别为 4，打印 error 日志
- critical：日志级别为 5，打印 critical 日志
- off：日志级别为 6，关闭日志打印

默认值：release 版本是 err，debug 版本是 debug

4.2.5.6 MACA_LAUNCH_MODE

描述：设置下发 GPU 任务时的刷缓存行为，目前在验证阶段。

可选值：

- 0：每个 GPU 任务完成后都会刷一次 L1 和 L2 缓存
- 1：每个 GPU 任务完成后只刷 L1 缓存，不刷 VL1S
- 2：每个 GPU 任务完成后只刷 SL1 缓存，不刷 VL1 和 L2

默认值：0，后续大规模测试通过后会默认配置为 1

4.2.5.7 MACA_MPS_MODE

描述：控制多进程场景下对 GPU 硬件 Queue 的调度。

可选值：

- 0：每个进程独占其申请到的 GPU 硬件 Queue，其它进程可以通过其它可用的 GPU 硬件 Queue 同时向 GPU 提交工作
- 1：允许多个进程共享同一个 GPU 硬件 Queue，进程通过一个或多个共享的 GPU 硬件 Queue 向 GPU 提交工作

默认值：0

4.2.5.8 MACA_DIRECT_DISPATCH

描述：配置 stream 下发任务时驱动侧的行为。

可选值：

- 0：每个 Stream 会额外创建一个线程，管理该 Stream 上的任务并提交到硬件执行
- 1：Stream 上的任务由 application 线程直接管理并提交到硬件去执行，多 Stream 场景下 CPU 负载较低

默认值：默认值为 0，后续经大规模验证后会将默认值配置为 1

4.2.6 工具相关

主要针对 MCCL test 和 Transferbench 等工具的环境变量。

4.2.6.1 MX_TRACER_ENABLED_MCPTI

描述：MCCL test 中用于控制是否进行 MCCL Kernel 时长度量。

可选值：

- ON：进行 MCCL Kernel 时长度量
- OFF：不进行 MCCL Kernel 时长度量

默认值：OFF

4.2.6.2 MCCL_OPTIMIZATION_A2A

描述：MCCL test 中用于控制是否开启 All to All 优化，该优化通过调用 `mcc1AllToAll` 实现。

可选值：

- 0：不开启优化
- 1：开启优化

默认值：0

4.2.6.3 P2P_MODE

描述：Transferbench 中用于控制 P2P 是单向或者双向测试。

可选值：

- 0：只进行单向 P2P 测试
- 1：分别进行单向和双向测试

默认值：0

5 附录

5.1 调试信息

5.1.1 共享内存

为了进程或线程间通信，MCCL 会在 `/dev/shm` 中创建共享内存。因而操作系统对共享内存的限制需要相应地增加。如果共享内存不够，MCCL 会在初始化时失败。增加共享内存大小可参考操作系统相关文档。

Docker 容器的默认配置会限制共享内存和固页内存的大小。当在 Docker 容器中运行 MCCL 应用程序时，需调整共享内存大小以确保程序可以成功运行。例如，在 Docker 运行时可以添加如下命令：

```
--shm-size=1g --ulimit memlock=-1
```

5.2 术语/缩略语

术语/缩略语	全称	说明
Docker		一个开源的应用容器引擎
MCCL	Metax Collective Communications Library	沐曦提供 GPU 间通信原语的库
MetaXLink		沐曦 GPU D2D 接口总线
MPI	Message Passing Interface	消息传递接口
MXMACA	MetaX Advanced Compute Architecture	沐曦推出的 GPU 软件栈，包含了沐曦 GPU 的底层驱动、编译器、数学库及整套软件工具套件
socket		网络编程标准接口，套接字